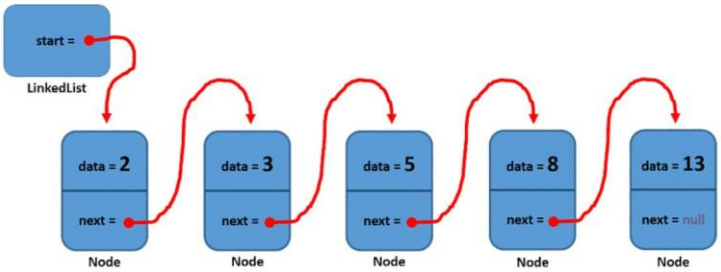
Struktur Data

LINKED LIST



1

Definisi

Linked List
Double Linked List
Circular Linked List

Objek Pembelajaran

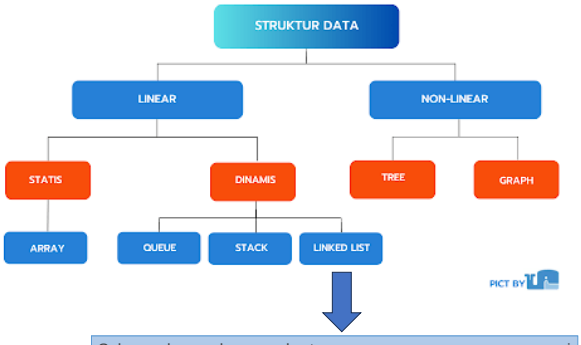
- Memahami definisi, jenis, representasi, operasi, dan penerapan struktur data Linked List
- Mampu memilih struktur data Linked List yang tepat untuk merepresentasikan data.
- Mampu menyelesaikan masalah dengan menggunakan struktur data Linked List secara efektif dan efisien.
- Meningkatkan kemampuan bernalar kritis, kreatif, mandiri, dan bergotong royong.

Keywords

Algoritma, Struktur data, linked list, double linked list, circular linked list

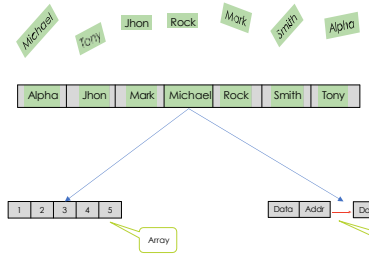
2

DEFINISI



Sekumpulan elemen bertype sama, yang mempunyai "keterurutan" tertentu, dan setiap elemennya terdiri dari dua bagian, yaitu informasi mengenai elemennya, dan informasi mengenai alamat elemen suksesornya (Liem, 2008).

Case → Susun nama mahasiswa berikut berdasarkan huruf pertama pada nama :




```
Type ElmtList : < Info : InfoType, Next : address >
```



Alamat yang sudah didefinisikan **Nil** disebut sudah **"di-alokasi"**. Didefinisikan suatu konstanta **Nil** yang artinya alamat yang tidak terdefinisi.

InfoType adalah sebuah type terdefinisi yang menyimpan informasi sebuah elemen list

Address "alamat" dari elemen berikutnya (suksesor)

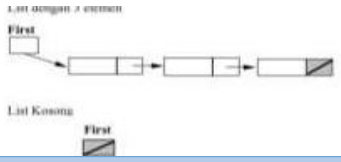
First Element (elemen pertama list)

3

Konsep List Linier

Sebuah list linier dikenali :

1. **Elemen pertamanya**, biasanya melalui alamat pertama yang disebut : **First**
2. Alamat **elemen berikutnya** (suksesor), jika kita mengetahui alamat sebuah elemen, yang dapat diakses memalui informasi NEXT. NEXT mungkin secara **eksplisit** (seperti contoh diatas) atau secara **implisit** yaitu lewat kalkulasi atau fungsi suksesor
3. Setiap elemen mempunyai **alamat**, yaitu tempat elemen disimpan dan diacu. Untuk mengacu sebuah elemen, alamat harus terdefinisi. Dengan lamat tersebut informasi yang tersimpan pada elemen list dapat diakses.
4. **Elemen terakhirnya**, ada berbagai cara untuk mengenali elemen akhir.



List Kosong

Jika L adalah List, P adalah address:
Alamat elemen pertama list L dapat diacu dengan notasi :
Fisrt (L)
Elemen yang diacu oleh P dapat dikonsultasi indormasinya dengan notasi selektor :
Info (P) Next (P)
Beberapa definisi :
•List L adalah list kosong, jika **First (L) = Nil**
•Elemen terakhir dikenali, misalnya jika Last adalah elemen terahir, maka **Next (Last) = Nil**

4

Skema Traversal untuk List Linier

List adalah koleksi objek yang terdiri dari sekumpulan elemen. Seringkali diperlukan proses terhadap setiap elemen list dengan cara yang sama. Karena itu salah satu primitif operasi konsultasi dasar pada struktur list adalah **Traversal**, yaitu "mengunjungi" setiap elemen list untuk di proses

```
procedure SKEMAListTraversal1(input L: list)
{I.S. List L terdefinisi, mungkin kosong}
{F.S. semua elemen List L "dikunjungi" dan telah diproses}
{Traversal sebuah list linier. Dengan MARK, tanpa pemrosesan khusus pada list
kosong}
KAMUS LOKAL
 P: address   {address untuk traversal, type terdefinisi}
 procedure Proses (input P: address)     {Pemrosesan elemen ber-address P}
 procedure Inisialisasi     {aksi sebelum proses dilakukan}
 procedure Terminasi        {aksi sesudah semua proses elemen selesai}
ALGORITMA
   Inisialisasi
   P ← First(L)
   while (P ≠ Nil) do
         proses(P)
         P ← Next(P)
   Terminasi
```

```
procedure SKEMAListTraversal2(input L: list)
{I.S. List L terdefinisi, mungkin kosong}
{F.S. semua elemen List L "dikunjungi" dan telah diproses}
{Traversal sebuah list linier yang diidentifikasi oleh elemen pertamanya L}
{Skema sekuensial dengan MARK dan pemrosesan khusus pada list kosong}
KAMUS LOKAL
 P: address   {address untuk traversal, type terdefinisi}
 procedure Proses (input P: address)     {Pemrosesan elemen ber-address P}
 procedure Inisialisasi     {aksi sebelum proses dilakukan}
 procedure Terminasi        {aksi sesudah semua proses elemen selesai}
ALGORITMA
     if First (L) = Nil then
        output ("List kosong")
     else
       Inisialisasi
       P ← First(L)
       repeat
             proses(P)
             P ← Next(P)
       Until (P = Nil)
     Terminasi
```

```
procedure SKEMAListTraversal3(input L: list)
{I.S. List L terdefinisi, tidak kosong, minimal mengandung satu elemen}
{F.S. semua elemen List L "dikunjungi" dan telah diproses}
{Skema sekuensial dengan MARK tidak ada list kosong karena tanpa MARK}
KAMUS LOKAL
 P: address   {address untuk traversal, type terdefinisi}
 procedure Proses (input P: address)     {Pemrosesan elemen ber-address P}
 procedure Inisialisasi     {aksi sebelum proses dilakukan}
 procedure Terminasi        {aksi sesudah semua proses elemen selesai}
ALGORITMA
      Inisialisasi
      P ← First(L)
      iterate
              proses(P)
      stop (Next(P) = Nil)
              P ← Next(P)
      Terminasi
```



5

Skema Sequential Search untuk List Linier

Proses pencarian suatu elemen list adalah primitif yang seringkali didefinisikan pada struktur list. Pencarian dapat berdasarkan nilai, atau berdasarkan alamat.

```
procedure SKEMAListSearch1 (input L: list, input X : InfoType, output P : address,
output Found : boolean)
{I.S. List linier L sudah terdefinisi dan siap dikonsultasi, X terdefinisi}
{F.S. P : address pada pencarian berurutan, Dimana X ditemukan, P = Nil jika tidak
ditemukan}
{Found berharga true jika harga X yang dicari ditemukan, false jika tidak ditemukan}
{sequential search harga X pada sebuah list linier L}
{Elemen diperiksa dengan intruksi yang sama, versi dengan Boolean}
KAMUS LOKAL
ALGORITMA
   P ← First(L)
   Found ← false
   While (P ≠ Nil) and (not Found) do
    If (X = Info(P)) then
       Found ← true
    Else
       P ← Next (P)
   {P = Nil or found}
   {jika Found maka P = address dari harga yang dicari ditemukan}
   {jika not Found maka P = Nil}
```

```
procedure SKEMAListSearch2 (input L: list, input X : InfoType, output P : address, output Found
: boolean)
{I.S. List linier L sudah terdefinisi dan siap dikonsultasi, X terdefinisi}
{F.S. P : address pada pencarian berurutan, Dimana X ditemukan, P = Nil jika tidak ditemukan}
{Found berharga true jika harga X yang dicari ditemukan, false jika tidak ditemukan}
{sequential search harga X pada sebuah list linier L}
{Elemen terakhir diperiksa secara khusus, versi dengan Boolean}
KAMUS LOKAL
ALGORITMA
   {List linier L sudah terdefinisi dan siap dikonsultasi}
   P ← Nil
   Found ← false
   If (First(L) = Nil) then
       Output ("List Kosong")
   Else {First(L) ≠ Nil, suksesor elemen pertama ada}
       P ← First (L)
       While((Next(P) ≠ Nil) and (X ≠ Info(P)) do
          P ← Next(P)
       {Next(P) = Nil or X = Info(P)}
       If (X = Info(P)) then
           Found ← true
       Else
           P ← Nil
   {P = Nil or found}
   {jika Found maka P = address dari harga yang dicari ditemukan}
   {jika not Found maka P = Nil}
```

```
procedure SKEMAListSearch@(input L: list, input P: address, output Found: boolean)
{I.S. List linier L sudah terdefinisi dan siap dikonsultasi, X terdefinisi}
{F.S. Jika ada elemen list yang beralamat P,Found berharga true}
{sequential search address P pada sebuah list linier L}
{semua elemen diperiksa dengan intruksi yang sama}
KAMUS LOKAL
 Pt: address
ALGORITMA
 Pt ← First(L)
 Found ← false
 While (P ≠ Nil) and (not Found) do
       If (Pt = P) then
           Found ← true
       Else
           Pt ← Next(Pt)
  {Pt = Nil or found}
  {jika Found maka P = adalah elemen list}
```



6

Definisi fungsional list linier dan algoritmanya

Secara fungsional, pada sebuah list linier biasanya dilakukan **pembuatan**, **pembahan**, **pengubahan** atau **penghapusan** elemen atau proses terhadap keseluruhan list, yang dapat ditulis sebagai berikut:

```
Diberikan L, L1 dan L2 adalah list linier dengan elemen ElmtList
IsEmpty     : L → Boolean          {tes apakah list kosong}
CreateEmpty : → L                  {membentuk sebuah list linier kosong}
Insert      : ElmtList x L → L     {menyisipkan sebuah elemen ke dalam list}
Delete      : L → L x ElmtList     {menghapus sebuah elemen list}
UpdateList  : ElmtList x L → L     {mengubah informasi sebuah elemen list linier}
Concat      : L1 x L2 → L          {menyambung L1 dan L2}
```

Dari definisi fungsional tersebut, dapat diturunkan fungsi-fungsi lain.

```
function IsEmpty (L: List) → Boolean
{test apakah list L kosong, mengirimkan true jika list kosong, false jika tidak kosong}
KAMUS LOKAL
ALGORITMA
 → (First(L) = Nil
```

Pemeriksaan list kosong sangat penting, karena I.S dan F.S beberapa prosedur harus di definiskan berdasarkan keadaan list. Operasi list kosong seringkali membutuhkan penanganan khusus.

```
function CreateEmpty (output L: List)
{I.S. Sembarang}
{F.S. Terbentuk list L kosong: First(L) diinisialisasi dengan NIL}
KAMUS LOKAL
ALGORITMA
 (First(L) ← Nil
```

Pemeriksaan list kosong sangat penting, karena I.S dan F.S beberapa prosedur harus di definiskan berdasarkan keadaan list. Operasi list kosong seringkali membutuhkan penanganan khusus.

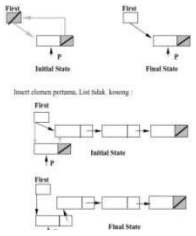
7

Definisi fungsional list linier dan algoritmanya

INSERT

Fungsi Insert (penyisipan) dapat menjadi penyisipan sebagai elemen pertama, setelah sebuah elemen P atau penyisipan menjadi elemen terakhir

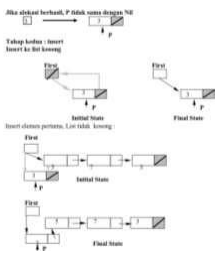
INSERT-FIRST (Diketahui Alamat)




```
Procedure InsertFirst (input/output L: List; input P: address)
{I.S. List L mungkin kosong, P sudah dialokasi, P ≠ Nil, Next(P)=Nil}
{F.S. P adalah elemen pertama pada List L}
{Insert sebuah elemen beralamat P sebagai elemen pertama list linier L yang mungkin kosong}
KAMUS LOKAL
ALGORITMA
 Next(P) ← First(L)
 First(L) ← P
```

Menambahkan sebuah elemen yang diketahui alamatnya sebagai elemen pertama list

INSERT-FIRST (Diketahui nilai)




```
Procedure InsertVFirst (input/output L: List; input InfoE: InfoType)
{I.S. List L mungkin kosong}
{F.S. sebuah elemen dialokasi dan menjadi elemen pertama list L, jika alokasi berhasil,
jika alokasi gagal list tetap seperti semula}
{Insert sebuah elemen sebagai elemen pertama list linier L yang mungkin kosong}
KAMUS LOKAL
function Alokasi (X: InfoType) → address
{Menghasilkan address yang dialokasi. Jika alokasi berhasil, Info(P)=InfoE, dan
Next(P)=Nil. Jika alokasi gagal, P=Nil}
P: address {Perhatikan: P adalah variable local}
ALGORITMA
 P ← Alokasi (InfoE)
 If (P ≠ Nil) then
    Next(P) ← First(L)
    First(L) ← P
```

Menambahkan sebuah elemen yang diketahui nilainya sebagai elemen pertama list

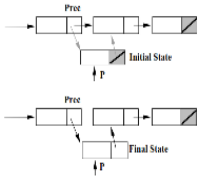
8

Definisi fungsional list linier dan algoritmanya

INSERT

Fungsi Insert (penyisipan) dapat menjadi penyisipan sebagai elemen pertama, setelah sebuah elemen P atau penyisipan menjadi elemen terakhir

INSERT-AFTER

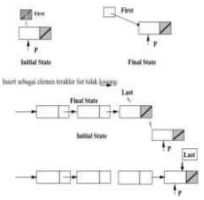



```
Procedure InsertAfter (input P, Prec: address)
{I.S. Prec adalah elemen list, Prec ≠ Nil, P sudah dialokasi, P ≠ Nil, Next(P) = Nil}
{F.S. P menjadi suksesor Prec}
{Insert sebuah elemen beralamat P pada List linier L}
KAMUS LOKAL

ALGORITMA
 Next(P) ← Next (Prec)
 Next(Prec) ← P
```

Menyisipkan sebuah elemen beralamat P setelah (sebagai suksesor dari) sebuah elemen list linier yang beralamat Prec

INSERT_LAST



```
Procedure InsertLast (input/ouput L: List, input P: address)
{I.S. List mungkin kosong, P sudah dialokasi, P ≠ Nil, Next(P) = Nil}
{F.S. P adalah elemen terakhir list L}
{Insert sebuah elemen beralamat P sebagai elemen terakhir dari list linier L yang mungkin
kosong}
KAMUS LOKAL
 Last: address {address untuk traversal, pada akhir address elemen terakhir}

ALGORITMA
 If IsEmpty(L) then   {insert sebagai elemen pertama}
      InsertFirst(L, P)
 else
      {Traversal list sampai address terakhir}
      {Bagaimana menghindari traversal list untuk mencapai Last?}
      Last ← First(L)
      While (Next(Last) ≠ Nil) do
          Last ← Next(Last)
      {Next(Last) = Nil, Last adalah elemen terakhir}
      {Insert P after Last}
       InsertAfter(P, Last)
```

Menyisipkan sebuah elemen beralamat P sebagai elemen terakhir sebuah list linier. Ada dua kemungkinan list kosong atau tidak kosong

```
Procedure InsertVLast (input/ouput L: List, input InfoE : InfoType)
{I.S. List mungkin kosong}
{F.S. Jika alokasi berhasil, InfoE adalah nilai elemen terkahir L}
{Jika alokasi gagal, maka F.S = I.S}
{Insert sebuah elemen beralamat P (jika alokasi berhasil) sebagai elemen terakhir dari
list linier L yang mungkin kosong}
KAMUS LOKAL
 function Alokasi (X: InfoType) → address
 {Menghasilkan address yang dialokasi. Jika alokasi berhasil, Info(P)=InfoE, dan
 Next(P)=Nil. Jika alokasi gagal, P=Nil}
  P: address {Perhatikan: P adalah variable local}

ALGORITMA
 P ← Alokasi(InfoE)
     if P ≠ Nil then  {insert sebagai elemen pertama}
      InsertLast(L,P)
```

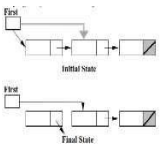
9

Definisi fungsional list linier dan algoritmanya

DELETE

Penghapusan elemen dapat merupakan elemen pertama, setelah sebuah address P atau penghapusan elemen terakhir. Operasi penghapusan dapat mengakibatkan list menjadi kosong. Jika list semula hanya terdiri dari satu elemen

DELETE-FIRST




```
Procedure DeletetFirst (input/output L: List; output P: address)
{I.S. List L tidak kosong, minimal 1 elemne, elemen pertama pasti ada}
{F.S. First 'maju', mungkin bernilai Nil(list menjadi kosong)}
{menghapus elemen pertama L, P adalah @elemen pertama L sebelum penghapusan, L yang baru
adalah Next(L)}
KAMUS LOKAL

ALGORITMA
 P ← First(L)
 First(L) ← Next(First(L))
 Next(P) ← Nil
 {perhatikan bahwa tetap benar jika list menjadi kosong}
```

Elemen yang dihapus dicatat alamatnya

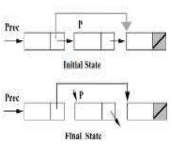
```
Procedure DelVFirst (input/output L: List; output E : InfoType)
{I.S. List L tidak kosong, minimal 1 elemen, elemen pertama pasti ada}
{F.S. menghapus elemen pertama L
E adalah nilai elemen pertama L sebelum penghapusan, L yang baru adalah Next(L)}
KAMUS LOKAL
procedure DeAlokasi (input P : address)
{I.S. P pernah dialokasi}
{F.S. Mengembalikan address yang pernah dialokasi. P =Nil}
P: address

ALGORITMA
 DeleteFirst(L, P)
 E ← Info(P)

 DeAlokasi(P)
```

elemen yang dihapus dicatat informasinya, dan Alamat elemen yang dihapus didealokasi

DELETE-AFTER




```
Procedure DeletetAfter (input Prec: address; output P: address)
{I.S. List tidak kosong, prec adalah elemen list, Next(Prec) ≠ Nil}
{F.S. Next(Prec), yaitu elemen beralamat P dihapus dari list, Next(P) = Nil}
{menghapus Suksesor prec, P adalah @suksesor prec sebelum penghapusan, Next(prec) yang
baru adalah suksesor dari suksesor prec sebelum penghapusan}
KAMUS LOKAL

ALGORITMA
 P ← Next(prec)
 Next(prec) ← Next(Next(prec))
 Next(prec) ← Nil
```

Penghapusan suksesor sebuah elemen

```
Procedure DeletetP (input/output L: list; input P: address)
{I.S. List L tidak kosong, P adalah elemen list L}
{F.S. menghapus P dari list L, P mungkin elemen pertama, 'tengah' atau terakhir}
KAMUS LOKAL
 Prec : address      {alamat predesesor P}

ALGORITMA
 {cari predesesor P}
 If (P = First(L) then       {delete list dengan satu elemen}
      DeleteFirst(L, P)
 else
      Prec ← First(L)
      While (Next(Prec) ≠ P) do
        Prec ← Next(Prec)
      {Next (Prec) = P, hapus P}
      DeleteAfter(Prec, P)
```

Penghapusan elemen beralamat P dapat dilakukan dengan mencari predesesor dari P, yaitu Alamat Prec. Memakai DeleteAfter(Prec, P)

10

Definisi fungsional list linier dan algoritmanya

DELETE

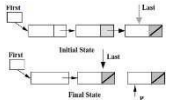
Penghapusan elemen dapat merupakan elemen pertama, setelah sebuah address P atau penghapusan elemen terakhir. Operasi penghapusan dapat mengakibatkan list menjadi kosong. Jika list semula hanya terdiri dari satu elemen

DELETE-LAST

List menjadi kosong



List tidak menjadi kosong




```
Procedure DeletetLast (input Firts: list; output P: address)
{I.S. List L tidak kosong, minimal mempunyai 1 elemen}
{F.S. P berisi alamat elemen yang dihapus, next(P) = Nil, list berkurang elemennya}
{Menghapus elemen terakhir dari list L, list mungkin menjadi kosong}
KAMUS LOKAL
  Last, PrecLast : address     {address untuk traversal, type terdefinisi pada akhirnya
                                last adalah alamat elemen terakhir dan PecLast adalah alamat
                                sebelum yang terakhir}
ALGORITMA
 {cari Last dan address sebelum Last}
 Last ← First(L)
 PrecLast ← Nil    {predesesor dari L tidak terdefinisi}
 While (Next(Last) ≠ Nil) do
      {Traversal list sampai @ terakhir}
      PrecLast ← Last
      Last ← Next(Last)
     {Next(Last) = Nil, last adalah elemen terakhir}
     {PrecLast = sebelum last}
     P ← Last
     If (Preclsat = Nil) then   {list dengan 2 elemen, jadi kosong}
         First(L) ← Nil
     else
         Next(Preclast) ← Nil
```

Menghapus elemen terakhir list dapat dilakukan jika Alamat dari elemen sebelum elemen terakhir diketahui. Persoalan selanjutnya menjadi DeleteAfter jika last bukan satu-satunya elemen list linier. Ada dua kasus, yaitu list menjadi kosong atau tidak.

Catatan :

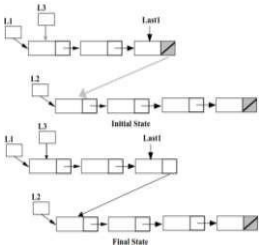
- Operasi *delete* yang tidak melakukan dealokasi address, ditawarkan jika address tersebut masih dibutuhkan. Misalnya elemen yang dihapus akan dijadikan anggota elemen list lain : dalam hal ini tidak perlu melakukan dealokasi, kemudian alokasi ulang Ketika akan ditambahkan ke list yang baru. Beberapa list dengan elemen sejenis seringkali harus dikelola dalam program; misalnya list mahasiswa per matakuliah.
- Operasi *delete* yang sekaligus melakukan dealokasi address, biasanya ditawarkan ke pengguna yang tidak perlu mengetahui 'keberadaan' address, dan yang hanya perlu memanipulasi informasi pada field Info

11

Definisi fungsional list linier dan algoritmanya

KONKATENASI DUA BUAH LIST LINIER

Konkatenasi atau menggabungkan dua list.




```
Procedure CONCAT (input L1, L2 : List; output L3 : List)
{I.S. L1 ≠ L2, L1 ≠ L3 dan L3 ≠ L2; L1 L2 mungkin kosong}
{F.S. L3 adalah hasil konkatenasi 'menyambung' dua buah list linier, L2 ditaruh dibelakang L1}
{Catatan : pelajari baik-baik algoritma berikut, apakah kedua list asal tetap dapat dikenali?}
KAMUS LOKAL
   Last1 : address {alamat elemen terakhir list pertama}
ALGORITMA
 CreateEmpty(L3) {Inisialisasi list hasil}
 If IsEmpty(L1) then
    First(L3) ← First(L2)
 else {Traversal list 1 sampai addresss terakhir, hubungkan last (L1) dengan first (L2)}
    First(L3) ← First(L1)
    Last1 ← First(L1
    While (Next(Last1) ≠ Nil) do
         Last1 ← Next(Last1)
        {Next(Last1) = Nil, last adalah elemen terakhir}
        Next(Last1) ← First(L2)
```

12

Implementasi Algoritma dan Bahasa

13